⚡ ESP32-C3 Cyber Keychain

The Complete Engineering & Coding Textbook

From Basic Electronics to Advanced Embedded C++ & BLE

📚 Table of Contents

🔌 Module 1: Electronics & Microcontrollers 101

Digital vs Analog Pins, PWM & The Classic LED Blink

💡 1. What is a Microcontroller (ESP32-C3)?

A microcontroller is a complete computer on a single chip containing:

- **Processor Core (CPU)**: Executes code instructions one by one.
- **RAM (Memory)**: Fast temporary workbench for variables and graphics.
- **Flash (Storage)**: Permanent hard drive where your compiled code lives.
- **GPIO (General Purpose Input/Output)**: Physical metal pins that can send or receive electricity to control displays, sensors, buttons, and LEDs.

⚡ 2. Digital Pins: 0 or 1 (Nothing in Between)

Digital pins only understand two electrical states:

- `HIGH` **(Logic 1)**: The pin connects to $3.3\ \mathrm{V}$.
- `LOW` **(Logic 0)**: The pin connects to $0\ \mathrm{V}$ **(Ground / GND)**.

Modes of a Digital Pin:

1. `OUTPUT`: The ESP32 sends electricity OUT (e.g., turning on an LED or screen backlight).
2. `INPUT`: The ESP32 listens to see if electricity is coming IN (e.g., button pressed or TTP223 touch sensor triggered).
3. `INPUT_PULLUP` **/** `INPUT_PULLDOWN`: Internal resistors inside the ESP32 that pull an idle pin safely to $3.3\mathrm{V}$ or $0\mathrm{V}$ so it doesn't float randomly when nothing is touching it.

📈 3. Analog Pins: Reading Continuous Voltages

- Digital pins only see ON/OFF. But real-world signals (like battery voltage) change smoothly from 0V to 3.3V.
- **ADC (Analog to Digital Converter)**: Converts a continuous voltage into a number between 0 and 4095 (on a 12-bit ADC).
  - $0\ \text{V} \rightarrow 0$
  - $1.65\ \text{V} \rightarrow 2047$
  - $3.3\ \text{V} \rightarrow 4095$
- We use this to read the LiPo battery level!

🌊 4. PWM (Pulse Width Modulation): Faking Analog

Microcontrollers cannot output 1.5V natively. So how do we dim a screen backlight from 0% to 100%?

- **The PWM Secret**: We switch the pin `HIGH` and `LOW` thousands of times per second ($44,100\ \text{Hz}$).
- **Duty Cycle**:
  - `25% Duty Cycle`: ON for 25% of the time, OFF for 75% $\rightarrow$ Looks like a dim 25% brightness to human eyes!
  - `100% Duty Cycle`: ON all the time $\rightarrow$ Full brightness!

```
PWM 25%:  [--]______[--]______[--]______ (Dim)
PWM 75%:  [------]__[------]__[------]__ (Bright)
```

🚨 5. The Classic "Blink an LED" Explained Line-by-Line

```
#define LED_PIN 2 // Connect LED to GPIO 2

// setup() runs ONCE when the microcontroller powers on
void setup() {
    pinMode(LED_PIN, OUTPUT); // Tell ESP32 that GPIO 2 will send electricity OUT
}

// loop() runs repeatedly FOREVER in an infinite cycle
void loop() {
    digitalWrite(LED_PIN, HIGH); // Turn on: Push 3.3V out -> LED glows!
    delay(1000);                 // Pause and wait for 1000 milliseconds (1 second)
    digitalWrite(LED_PIN, LOW);  // Turn off: Drop to 0V -> LED dark
    delay(1000);                 // Pause for 1 second
}
```

- Every Arduino program has **`setup()`** (initialization) and **`loop()`** (continuous execution).

💻 Module 2: Embedded C++ Programming

Headers, Macros, Types, Classes & DMA

1. Headers & Preprocessor Directives

```
#pragma once      // Tells the compiler: "Only include this file once" (prevents duplicate errors)
#include <Arduino.h> // Includes core Arduino functions like pinMode, millis(), delay()
#define PIN_TFT_SCL 8 // Text replacement: every time compiler sees PIN_TFT_SCL, it replaces it with 8
```

- **Header files (`.h`)**: Contain blueprints, settings, and declarations.
- **Source files (`.cpp`)**: Contain the actual logic and implementation code.
- **Why use `#define` instead of variables?** `#define` does not consume any RAM! The compiler replaces text before building the binary.

🔢 2. Fixed-Width Integer Types

In embedded systems, saving memory is critical. Standard `int` can be different sizes on different chips, so we use **fixed-width types**:

Type	Bits	Bytes	Range	Usage in Project
`uint8_t`	8	1	0 to 255	Brightness ($0 - 255$), Colors, Pin numbers
`uint16_t`	16	2	0 to $65,535$	**RGB565 Screen Colors** (240×240 pixels)
`uint32_t`	32	4	0 to $4,294,967,295$	Millisecond timestamps (`millis()`)
`float`	32	4	Decimals ($\pm 3.4 \times 10^{38}$)	Gaze coordinates, temperatures
`enum`	8-32	1-4	Named states (`MODE_CYBERPET`)	System modes & moods

📦 3. Classes and Object-Oriented Programming (OOP)

A **Class** is a cookie cutter; an **Object** is the actual cookie.

```
class RobotEyes {
private: // Hidden variables (internal organs)
    float currentX = 120;
    float currentY = 120;

public:  // Public functions (buttons anyone can press)
    void update() {
        // Draw the eyes
    }
    void setMood(PetMood mood) {
        // Change eye color
    }
};

RobotEyes myEyes; // Create an instance (object) of the class
myEyes.update();  // Call its function!
```

🚀 4. Direct Memory Access (DMA) & Double Buffering

- **The Problem (Screen Tearing)**: If you draw directly to the LCD screen, human eyes see horizontal tearing lines because the screen is drawing while you are clearing pixels.
- **The Solution (Double Buffering with a Sprite)**:
  1. We allocate a 240×240 block in ESP32 RAM called `canvas` (`LGFX_Sprite`).
  2. We draw circles, text, cats, and eyes onto `canvas` in RAM.
  3. We call `canvas.pushSprite(0,0);` $\rightarrow$ **DMA hardware** transfers all 57,600 pixels to the display via SPI at $40\ \mathrm{MHz}$ with zero CPU overhead!

📡 Module 3: Bluetooth Low Energy (BLE)

GATT Architecture, Services & Characteristics

📶 1. Classic Bluetooth vs Bluetooth Low Energy (BLE)

- **Classic Bluetooth (e.g. Headphones)**: Constantly streaming audio at 50–100mA. Drains small batteries in 2 hours.
- **BLE (Bluetooth Low Energy)**: Designed for small bursts of data. Sleeps in microamps and wakes up in milliseconds when your phone pushes a command.

🌳 2. The GATT Profile Architecture

BLE structures communication like a tree:

```
[ ESP32-C3 BLE Server: "CYBER_KEYCHAIN" ]
      │
      └── [ PRIMARY SERVICE (UUID: 4fafc201...) ]
                │
                ├── Characteristic 1: Mode Switch (Write)
                ├── Characteristic 2: Avatar Select (Write)
                ├── Characteristic 3: Custom Text (Write)
                ├── Characteristic 4: Time & Weather Sync (Write)
                └── Characteristic 5: Brightness Slider (Write)
```

- **UUID (Universally Unique Identifier)**: A unique 128-bit address (e.g. `4fafc201-1fb5-...`) that your phone uses to find the exact slider or button to talk to.

🔍 Module 4: Codebase Line-by-Line Breakdown

Every File, Every Function Explained

📜 File 1: `config.h`

```
#pragma once
#include <Arduino.h>

// SPI Display Pins mapped to ESP32-C3 GPIOs
#define PIN_TFT_SCL    8    // SPI Clock
#define PIN_TFT_SDA    10   // SPI MOSI Data
#define PIN_TFT_RES    5    // Display Reset
#define PIN_TFT_DC     3    // Data/Command select
#define PIN_TFT_CS     7    // Chip Select
#define PIN_TFT_BL     2    // Backlight PWM pin

#define PIN_TOUCH      1    // TTP223 Touch Signal Pin (RTC GPIO)

// Enum: Named states for our state machine
enum SystemMode {
    MODE_CYBERPET = 0,   // Virtual Pet
    MODE_ROBOT_EYES = 1, // Animated Eyes
    MODE_CYBER_HUD = 2,  // Sci-Fi Clock
    MODE_MATRIX_RAIN = 3,// Matrix Rain
    MODE_TEXT_SCROLL = 4 // Marquee Text
};
```

🎨 File 2: `display_setup.h`

Configures the **`LovyanGFX`** hardware driver:

```
auto cfg = _bus_instance.config();
cfg.spi_host = SPI2_HOST;
cfg.freq_write = 40000000; // 40MHz High-Speed SPI clock
cfg.pin_sclk = PIN_TFT_SCL; // GPIO 8
cfg.pin_mosi = PIN_TFT_SDA; // GPIO 10
cfg.pin_dc   = PIN_TFT_DC;  // GPIO 3
_bus_instance.config(cfg);
```

- Configures the ESP32-C3 hardware SPI engine to blast pixels at $40\ \mathrm{MHz}$.
- Sets panel size to 240×240, enables color inversion (`invert = true` for ST7789 IPS panels), and attaches PWM brightness control to GPIO 2.

👀 File 3: `robot_eyes.h` — Procedural Animation

```
// 1. Natural Gaze Tracking
if (now - lastGazeChange > 2500 && !isBlinking) {
    lastGazeChange = now;
    targetX = 120 + random(-25, 26); // Pick new horizontal target
    targetY = 120 + random(-15, 16); // Pick new vertical target
}

// 2. Smooth Interpolation (Easing)
currentX += (targetX - currentX) * 0.2f;
currentY += (targetY - currentY) * 0.2f;
currentH += (targetH - currentH) * 0.35f;

// 3. Draw Eyes onto double-buffer canvas
canvas.fillRoundRect(leftEyeX - (eyeWidth/2), eyeY, eyeWidth, currentH, eyeRadius, eyeColor);
canvas.fillRoundRect(rightEyeX - (eyeWidth/2), eyeY, eyeWidth, currentH, eyeRadius, eyeColor);
```

- Smoothly glides gaze toward targets and calculates eye heights dynamically.

🐱 File 4: `pet_engine.h` — Mood State Machine

```
// Draw Cute CyberCat with Breathing Animation
void drawCyberCat(int cx, int cy) {
    uint16_t visorCol = (mood == MOOD_ANGRY) ? 0xF800 : ((mood == MOOD_HAPPY) ? 0xF81F : 0x07FF);

    // Body & Head
    canvas.fillRoundRect(cx - 38, cy - 25, 76, 60, 24, 0xFFFF);
    canvas.fillCircle(cx, cy - 18, 36, 0xFFFF);

    if (mood == MOOD_HAPPY) {
        // Draw Heart Eyes
        canvas.fillCircle(cx - 16, cy - 18, 7, visorCol);
        canvas.fillCircle(cx - 6, cy - 18, 7, visorCol);
        canvas.fillTriangle(cx - 23, cy - 16, cx + 1, cy - 16, cx - 11, cy - 4, visorCol);
        // Blush Cheeks
        canvas.fillCircle(cx - 26, cy - 6, 6, 0xFA48);
        canvas.fillCircle(cx + 26, cy - 6, 6, 0xFA48);
    }
}
```

⏱️ File 5: `cyber_hud.h` — Sci-Fi Telemetry

```
// 1. Format Time into HH:MM:SS string
char timeStr[16];
snprintf(timeStr, sizeof(timeStr), "%02d:%02d:%02d", hours, minutes, seconds);
canvas.setTextColor(0x07E0, TFT_BLACK); // Neon Green
canvas.setTextSize(2);
canvas.drawCenterString(timeStr, 120, 48);

// 2. Animated Signal Sine-Waves
for (int x = 20; x < 220; x += 2) {
    float y1 = 175 + sin((x + wavePhase) * 0.08f) * 16.0f;
    float y2 = 175 + sin((x - wavePhase * 1.5f) * 0.05f) * 10.0f;
    canvas.drawPixel(x, (int)y1, 0x07E0); // Green wave
    canvas.drawPixel(x, (int)y2, 0x07FF); // Cyan wave
}
```

File 6: `matrix_rain.h` — 60 FPS Digital Rain

```
for (int i = 0; i < MATRIX_COLS; i++) {
    int x = i * 15 + 4;
    // Draw 8 fading green trailing characters
    for (int t = 1; t <= 8; t++) {
        int trailY = yPos[i] - (t * 14);
        if (trailY >= 0 && trailY < 240) {
            uint8_t greenDim = 255 - (t * 28); // Fade brightness
            canvas.setTextColor(canvas.color565(0, greenDim, 0), TFT_BLACK);
            canvas.drawChar(getRandomChar(), x, trailY);
        }
    }
    // Draw bright glowing head character in White
    canvas.setTextColor(TFT_WHITE, TFT_BLACK);
    canvas.drawChar(characters[i], x, yPos[i]);
    yPos[i] += speed[i]; // Move down screen
}
```

🧠 File 7: `main.cpp` — Gesture Engine & Sleep

```
void processTouch() {
    bool rawTouch = digitalRead(PIN_TOUCH) == HIGH;
    uint32_t now = millis();

    // Detect Hold / Stroke (> 1.2 seconds) -> HAPPY MOOD!
    if (isTouching && (now - touchStartTime > 1200)) {
        cyberPet.setMood(MOOD_HAPPY);
        robotEyes.setMood(MOOD_HAPPY);
    }
    // Detect Rage Tapping (3+ rapid taps) -> ANGRY MOOD!
    if (!isTouching && tapCount >= 3) {
        cyberPet.setMood(MOOD_ANGRY);
        robotEyes.setMood(MOOD_ANGRY);
        tapCount = 0;
    }
}
```

File 8: `web/index.html` — Web Bluetooth

```
// Scan & Connect over Web Bluetooth
async function connectBLE() {
    bleDevice = await navigator.bluetooth.requestDevice({
        filters: [{ name: 'CYBER_KEYCHAIN' }],
        optionalServices: [SERVICE_UUID]
    });
    gattServer = await bleDevice.gatt.connect();
    const service = await gattServer.getPrimaryService(SERVICE_UUID);
    charMode = await service.getCharacteristic(CHAR_MODE_UUID);
}

// Switch Mode Function
async function setMode(modeNum) {
    const data = new TextEncoder().encode(modeNum.toString());
    await charMode.writeValue(data); // Transmit over the air!
}
```

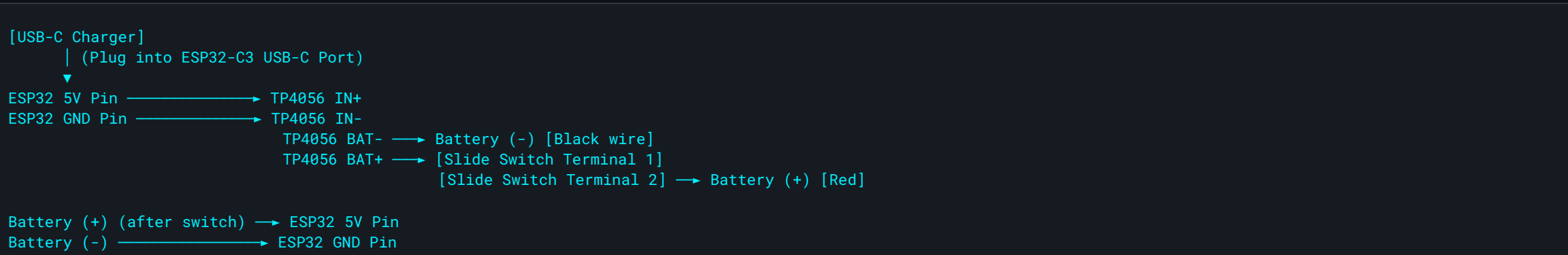
🔌 Module 5: Minimalist Wiring Blueprint

Zero Clutter • Exact Pin Connections

🗺️ Master Pin-to-Pin Table

Component & Pin	Connects to ESP32 Pin	Purpose
Display GND	ESP32 `GND`	Ground Reference
Display VCC	ESP32 `3.3V`	3.3V Logic Power
Display SCL	ESP32 `GPIO 8`	Hardware SPI Clock
Display SDA	ESP32 `GPIO 10`	Hardware SPI Data (MOSI)
Display RES	ESP32 `GPIO 5`	Hardware Reset
Display DC	ESP32 `GPIO 3`	Data / Command Selector
Display BLK	ESP32 `GPIO 2`	Backlight PWM Brightness
TTP223 VCC & GND	ESP32 `3.3V` & `GND`	Touch Sensor Power
TTP223 I/O Signal	ESP32 `GPIO 1`	Deep Sleep RTC Wakeup Interrupt

🔋 Power & Battery Routing (1 Resistor Total)



- **Resistor Mod**: Replace resistor `122` ($1.2\text{k}\Omega$) on the TP4056 with a $4.7\text{k}\Omega$ **resistor** so charging current drops from $1000\text{mA} \rightarrow 255\text{mA}$ safely!

❓ Module 6: The 32 Comprehensive FAQs

Every Doubt Answered Completely

🛠️ FAQs 1 – 8 (Soldering & Hardware)

1. **Can I build this without a custom PCB?** Yes, the green perfboard with wire bridges is standard and gives a retro cyberpunk aesthetic.
2. **What wires should I use?** 30 AWG flexible silicone wire or enameled copper magnet wire.
3. **Why does my screen stay black?** Check that `SCL` is on GPIO8, `SDA` is on GPIO10, and `BLK` is connected to GPIO2.
4. **Is 3.3V safe for the 1.3" display?** Yes, ST7789 displays run natively on 3.3V logic.
5. **Do I need pull-up resistors on SPI pins?** No, SPI is actively driven push-pull by the ESP32 hardware.
6. **How do I cut the perfboard neatly?** Score both sides with a utility knife along a row of holes, then snap it over a table edge.
7. **Will the solder joints short out on my bag?** Coat the back of the perfboard with 1–2 coats of clear nail polish to insulate it.
8. **Can I attach a metal touch bolt?** Yes, solder a brass screw or copper washer to the TTP223 touch pad to act as a touch trigger.

🔋 FAQs 9 – 16 (Battery & Charging)

9. **Why MUST I change the resistor on the TP4056?** Default is 1000mA (too high for 200mAh). 4.7kΩ makes it safe at ~250mA.
10. **Can I leave it charging overnight?** Yes, the TP4056 automatically cuts off current when the battery hits 4.20V.
11. **How long will a 200mAh battery last?** In deep sleep mode with ~25 wakeups/day, it easily lasts 2 to 3 weeks.
12. **What happens when the battery gets empty?** The DW01 protection chip on the TP4056 shuts off power before the battery dips below 2.5V.
13. **Can I use a larger 500mAh battery?** Yes, it will last 2.5x longer, just takes slightly more physical room.
14. **Why connect the battery to 5V instead of 3.3V?** A full LiPo outputs 4.2V. The 5V pin feeds through the onboard regulator to drop it safely to 3.3V.
15. **Does the slide switch disable charging?** No, the battery will still charge when plugged into USB regardless of switch state.
16. **Can I play with it while charging?** Yes, it runs normally while connected to USB.

💻 FAQs 17 – 24 (Code & PlatformIO)

17. **How do I upload with PlatformIO?** Open the folder in VS Code, connect USB, and click the Upload arrow icon.
18. **What if upload fails?** Hold the `BOOT` button on SuperMini, plug in USB, click Upload, then release once flashing starts.
19. **Can I use Arduino IDE?** Yes, install `LovyanGFX` and `NimBLE-Arduino` from Library Manager.
20. **Why LovyanGFX instead of TFT_eSPI?** LovyanGFX is faster on ESP32-C3, supports dynamic backlight control, and doesn't require external header setup.
21. **How do I change the default startup mode?** Change `currentMode = MODE_CYBERPET;` in `main.cpp` to any other mode.
22. **Can I add custom animations?** Yes, add new sprite drawing routines into `include/pet_engine.h` using standard shape functions.
23. **What baud rate for Serial monitor?** 115200 baud.
24. **Does the ESP32 remember code when battery dies?** Yes, compiled firmware stays in Flash memory permanently.

📶 FAQs 25 – 32 (Bluetooth & Web Remote)

25. **Do I need internet for Web BLE?** No, you can save `index.html` on your phone storage and open it offline.
26. **How do I use Web BLE on iPhone?** Use the free **Bluefy** app from the App Store (Safari blocks Web Bluetooth).
27. **How do I use Web BLE on Android?** Open `index.html` directly in standard **Google Chrome**.
28. **What is the Bluetooth range?** Around 10 to 15 meters line-of-sight.
29. **Can someone else hijack my keychain?** No, once your phone connects, the BLE server stops advertising and locks to your phone.
30. **How do I change the Bluetooth name?** Change `"CYBER_KEYCHAIN"` in `NimBLEDevice::init()` in `main.cpp`.
31. **Can I display live Spotify songs?** Yes, a script on your phone can push song titles via the text characteristic.
32. **How do I host the Web BLE dashboard online?** Upload `index.html` to a free GitHub repository and enable **GitHub Pages** for a free public URL!

🚀 Ready to Build!

- 📄 **Markdown Document**: `presentation.md`
- 🖥️ **Interactive HTML Slides**: `presentation.html`
- 📕 **Printable Document**: `presentation.pdf`

⚡ **Happy Making & Hacking!** ⚡